

מדריך Deployment — העברת

לומדת ענף 71

איך להעביר את הלומדה לכל מקום בעולם

תוכן עניינים

1. [סקירה כללית](#)
 2. [דרישות מקדימות](#)
 3. [Scenario 1: Cloud Deployment \(עם גישה לרשת\)](#)
 4. [Scenario 2: Local Network Deployment \(ללא אינטרנט\)](#)
 5. [Backup & Migration](#)
 6. [Troubleshooting](#)
-

סקירה כללית

שלוש אפשרויות Deployment:

שלוש דרכים להפעיל לומדה

1 CLOUD (גישה לרשת אינטרנט)

- └ AWS, Heroku, DigitalOcean, Linode
- └ יותר נגיש מחו"ל +
- └ קל להתקנה +
- └ עלויות חודשיות -

2 LOCAL NETWORK (ללא אינטרנט, רק רשת מקומית)

- └ Server / בתוך הבית / משרד
- └ ללא עלויות חודשיות +
- └ שליטה מלאה +
- └ צריך שרת פיזי -

3 HYBRID (Cloud + Local)

- └ Cloud, לגישה מרחוק
- └ Local, לנתונים
- └ הטוב משניהם +
- └ מורכב יותר -

דרישות מקדימות

סיוג על קבצים ודאטה

קבצים שצריך להעביר:

```
rf-learning-system/
├── backend/
│   ├── src/                # קוד
│   ├── uploads/           # קבצים שהעלו
│   ├── package.json
│   ├── tsconfig.json
│   └── .env                # משתנים סביבה (חשוב!)
├── frontend/
│   ├── src/                # קוד
│   ├── package.json
│   ├── vite.config.ts
│   └── dist/               # Build מוכן
└── database/
    ├── rf_learning.sql     # Backup של DB
    └── restore.sql         # Script להשחזור
```

כלים שתצטרך:

ל-Cloud:

- חשבון Cloud (AWS, Heroku, DigitalOcean וכו') 🌐
- SSH keys 🔑
- Docker (optional but recommended) 🐳

ל-Local Network:

- שרת פיזי (או NAS) 💻

PostgreSQL installed  •

Node.js installed  •

Ethernet כבל  •

עם) Scenario 1: Cloud Deployment

גישת לרשת

דרך א: DigitalOcean (הפשוטה ביותר)

שלב 1: הגדרה ב-DigitalOcean

1. DigitalOcean-הרשמה ל (www.digitalocean.com)
2. חדש Droplet צור:
 - Image: Ubuntu 22.04
 - Size: Basic \$5/month (כאפי לתחילה)
 - Region: בקרוב ליוצרים
 - Authentication: SSH Key
3. כמו: IP 192.168.1.100 אחרי יצירה, תקבל

שלב 2: Prepare Backend for Cloud

1. צור `.env` :Production

```
# backend/ בתיקיית .env
NODE_ENV=production
SERVER_PORT=3000

DB_HOST=your-digitalocean-db-ip
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=secure-password-here
DB_NAME=rf_learning

JWT_SECRET=your-super-secret-key-change-this
UPLOAD_DIR=/var/uploads
```

2. Build Backend :

```
cd backend
npm install --production # Production dependencies only
npm run build            # If you have a build script
```

.3 Package for Upload

```
# Create tar archive
tar -czf backend.tar.gz \
  backend/src \
  backend/node_modules \
  backend/package.json \
  backend/.env
```

שלב 3: Prepare Frontend for Cloud

.1 Build Frontend

```
cd frontend
npm install
npm run build
# עם קבצים סטטיים dist/ יוצר תיקיית
```

.2 Package Frontend

```
tar -czf frontend-dist.tar.gz frontend/dist/
```

שלב 4: Upload to DigitalOcean

.1 Connect via SSH

```
ssh root@your-digitalocean-ip
```

.2 Install Requirements


```
# Update system
apt-get update && apt-get upgrade -y

# Install Node.js
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs

# Install PostgreSQL
apt-get install -y postgresql postgresql-contrib

# Install Nginx (as reverse proxy)
apt-get install -y nginx

# Install PM2 (for background processes)
npm install -g pm2
```

:Extract Files .3

```
# Create directories
mkdir -p /var/www/rf-learning
mkdir -p /var/uploads

# Upload files using SCP from your computer:
scp backend.tar.gz root@your-ip:/var/www/
scp frontend-dist.tar.gz root@your-ip:/var/www/

# Extract on server
cd /var/www
tar -xzf backend.tar.gz
tar -xzf frontend-dist.tar.gz
```

:Setup Database .4

```
# Enter PostgreSQL
sudo -u postgres psql

# Create database
CREATE DATABASE rf_learning;

# Create user with password
CREATE USER postgres WITH PASSWORD 'your-secure-password';

# Grant privileges
GRANT ALL PRIVILEGES ON DATABASE rf_learning TO postgres;

# Exit
\q

# Restore database from backup (if you have one)
psql -U postgres -d rf_learning < rf_learning_backup.sql
```

:Configure Nginx as Reverse Proxy .5

```
# Edit Nginx config
nano /etc/nginx/sites-available/default
```

:Replace content with

```

upstream backend {
    server 127.0.0.1:3000;
}

server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;
    root /var/www/rf-learning/frontend/dist;

    # Frontend routing (SPA)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Backend API
    location /api/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

    # File uploads
    location /uploads/ {
        alias /var/uploads/;
    }
}

```

:Test Nginx

```
nginx -t
systemctl reload nginx
```

:Start Backend with PM2 .6

```
cd /var/www/backend
npm install # Install remaining dependencies

# Start with PM2
pm2 start src/server.ts --name "rf-learning-backend"

# Save PM2 process list
pm2 save

# Restart on reboot
pm2 startup
```

:Test Everything .7

```
# Check if backend is running
curl http://localhost:3000/api/courses

# Check if frontend is accessible
curl http://localhost
```

שלב 5: Domain & SSL (Optional but Recommended)

:Point Domain .1

שם דומיין שלך DNS בקטע:

A Record: your-domain.com → your-digitalocean-ip

:Setup SSL with Let's Encrypt .2

```
apt-get install certbot python3-certbot-nginx  
certbot --nginx -d your-domain.com  
  
# Auto-renewal  
systemctl enable certbot.timer
```

URL הנתקלה בחיים:

```
https://your-domain.com      → Frontend  
https://your-domain.com/api/ → Backend
```

דרך ב: Heroku (ללא DevOps)

שלב 1: Heroku Setup

```
# Install Heroku CLI
# Windows: Download from heroku.com/download

# Login
heroku login

# Create app
heroku create my-rf-learning
```

שלב 2: Database Setup

```
# Add PostgreSQL addon
heroku addons:create heroku-postgresql:hobby-dev -a my-rf-learning

# Get connection string
heroku config -a my-rf-learning

# Copy DATABASE_URL value
```

שלב 3: Deploy

:Create Procfile in root

```
web: cd backend && npm start
```

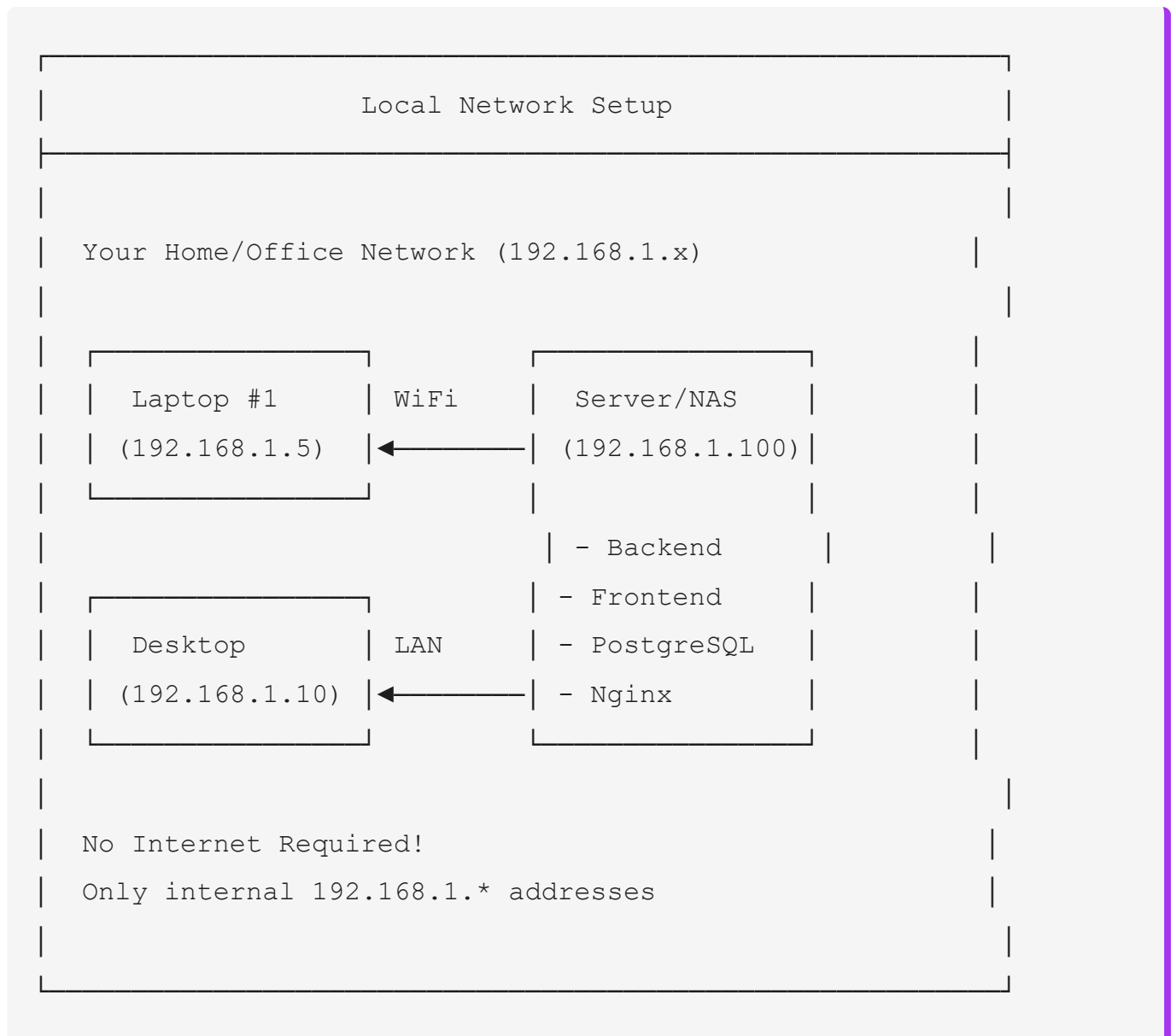
:Push to Heroku

```
git push heroku main
```

That's it! Your app is live at: <https://my-rf-learning.herokuapp.com>

Scenario 2: Local Network (ללא אינטרנט) Deployment

סקירה: איך זה עובד



שלב 1: בחר שרת מקומי

אפשרות 1: Old Computer as Server

תמונה:

Old Win7 PC / Old Mac → Repurpose as Server

יתרונות:

- חינם ✓
- בקרה מלאה ✓
- צריך משהו שרץ 24/7 ✗

צעדים:

```
# Install Node.js & PostgreSQL on the server computer
# Copy backend/ and frontend/ directories
# Start with npm run dev or PM2
```

אפשרות 2: Raspberry Pi / Mini PC

מחיר: \$35-100

צריכת חשמל: 5-15W (זול!)

מה שתצטרך:

- Raspberry Pi 4 (4GB RAM) = \$55
- מברטס SD 64GB = \$15
- Ethernet cable
- Power supply

סה"כ: ~\$80 יחד עם צריכה חשמלית נמוכה

אפשרות 3: NAS (Network Attached Storage)

מחיר: \$200-500

דוגמה: Synology DS220j

יתרונות:

- Built-in RAID (backup אוטומטי) ✓
- Web interface ✓
- Stable & reliable ✓
- יקר ✗

שלב 2: Setup Server (Ubuntu/Raspberry Pi OS)

:Installation Steps

```
# 1. Update system
sudo apt-get update && sudo apt-get upgrade -y

# 2. Install Node.js
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs

# 3. Install PostgreSQL
sudo apt-get install -y postgresql postgresql-contrib

# 4. Install Nginx
sudo apt-get install -y nginx

# 5. Install PM2
sudo npm install -g pm2

# 6. Clone/Copy project
git clone <your-repo> /opt/rf-learning
# OR
sudo cp -r /path/to/rf-learning /opt/rf-learning
```

שלב 3: Configure Database

```
# Start PostgreSQL
sudo systemctl start postgresql

# Switch to postgres user
sudo -u postgres psql

# Create database
CREATE DATABASE rf_learning;

# Set password
ALTER USER postgres WITH PASSWORD 'your-local-password';

# Exit
\q

# Restore from backup (if available)
psql -U postgres -d rf_learning < rf_learning_backup.sql
```

שלב 4: Setup Backend

:Create .env file

```
sudo nano /opt/rf-learning/backend/.env
```

:Content

```
NODE_ENV=production
SERVER_PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=your-local-password
DB_NAME=rf_learning

JWT_SECRET=change-this-to-something-secure
UPLOAD_DIR=/opt/rf-learning/backend/uploads
```

:Install & Start

```
cd /opt/rf-learning/backend
npm install
pm2 start src/server.ts --name "rf-learning-backend"
pm2 save
pm2 startup
```

שלב 5: Build Frontend

```
cd /opt/rf-learning/frontend
npm install
npm run build

# Copy to Nginx
sudo cp -r dist/* /var/www/html/
```

שלב 6: Configure Nginx

```
sudo nano /etc/nginx/sites-available/default
```

Same config as Cloud setup above

שלב 7: Access from Network

:Find Server IP

```
hostname -I
# Output: 192.168.1.100
```

:Access from any device on network

```
http://192.168.1.100    (Frontend)
http://192.168.1.100/api/courses    (API)
```

שלב 8: Enable Wake-on-LAN (Optional)

Goal: Turn on server remotely from another computer

:Setup

```
# BIOS: Enable Wake-on-LAN
# Linux: sudo apt-get install wakeonlan

# From laptop:
wakeonlan 00:11:22:33:44:55 # Server MAC address
```

Backup & Migration

איך לעשות Backup של הכל

1. Backup Database

```
# From server
pg_dump -U postgres rf_learning > rf_learning_backup.sql

# Compress for storage
gzip rf_learning_backup.sql
# Output: rf_learning_backup.sql.gz
```

2. Backup Uploads Folder

```
tar -czf uploads_backup.tar.gz /opt/rf-learning/backend/uploads/
```

3. Backup Everything

```
# Create full backup
tar -czf rf-learning-full-backup-$(date +%Y%m%d).tar.gz \
/opt/rf-learning/ \
rf_learning_backup.sql
```

4. Store Backup

אפשרויות:

1. External Hard Drive

- Connect to server
- Copy backup files
- Disconnect & store

2. Cloud Storage

- Dropbox, Google Drive
- AWS S3
- Synology C2

3. NAS

- If you have one

איך להחזיר מ-Backup

Scenario: Moving to New Server

```
# 1. On new server, setup everything (see above)

# 2. Download backup
scp user@old-server:rf_learning_backup.sql.gz .
gunzip rf_learning_backup.sql.gz

# 3. Restore database
psql -U postgres -d rf_learning < rf_learning_backup.sql

# 4. Copy uploads
scp -r user@old-server:/opt/rf-learning/backend/uploads/* \
    /opt/rf-learning/backend/uploads/

# 5. Restart backend
pm2 restart rf-learning-backend
```

Troubleshooting

Cloud Issues

"Backend not connecting to Database"

```
# Check if DB is up
psql -h your-db-ip -U postgres -d rf_learning

# Check backend logs
pm2 logs rf-learning-backend

# Verify .env has correct DB_HOST
cat backend/.env | grep DB_HOST
```

"Frontend shows 404 on page refresh"

```
# Nginx needs SPA config
# Make sure your nginx config has:
location / {
    try_files $uri $uri/ /index.html;
}
```

"SSL Certificate Error"

```
# Renew certificate
sudo certbot renew

# Force renewal
sudo certbot renew --force-renewal
```

Local Network Issues

"Can't access server from laptop"

```
# 1. Check server is on same network
ping 192.168.1.100

# 2. Check if Nginx is running
sudo systemctl status nginx

# 3. Check firewall
sudo ufw allow 80
sudo ufw allow 443
sudo ufw allow 3000
```

"Database connection refused"

```
# Check if PostgreSQL is running
sudo systemctl status postgresql

# Check if listening on localhost
sudo -u postgres psql -c "SELECT version();"

```

"PM2 not restarting after reboot"

```
# Reinstall startup
pm2 save
pm2 startup
sudo env PATH=$PATH:/usr/bin
/usr/local/lib/node_modules/pm2/bin/pm2 startup
```

Comparison Table

Cloud vs Local Network

Local Network	Cloud	Feature
לא צריך	חובה	צריכת אינטרנט
ללא	\$5-20	עלות חודשית
לא ✗	כן ✓	גישה מחו"ל
מלאה	מוגבלת	בקרה
בינונית	גבוהה	סטיביליות
צריך IT ⚠	קל ✓	יציאה
לא כל כך	מומלץ ✓	עבור תחילה
מומלץ ✓	מומלץ	עבור חברה

Deployment לפני — Checklist

Before Going Live

- ☐ Database backup created
- ☐ .env file with secure secrets
- ☐ Frontend built (npm run build)
- ☐ Backend dependencies installed
- ☐ SSL certificate (if using domain)
- ☐ Nginx/reverse proxy configured
- ☐ PM2/supervisor setup for auto-restart
- ☐ Firewall rules configured
- ☐ IP whitelisting (if needed)
- ☐ Email notifications for errors
- ☐ Monitoring setup
- ☐ Documentation for admin
- ☐ Training for users

After Going Live

- ☐ Daily backup schedule
 - ☐ Weekly database verification
 - ☐ Monitor server logs
 - ☐ Update Node.js/PostgreSQL monthly
 - ☐ Security patches ASAP
 - ☐ User feedback collection
 - ☐ Performance monitoring
-

Quick Start Scripts

Cloud Quick Deploy (DigitalOcean)

save as: [deploy-cloud.sh](#)

```
#!/bin/bash

echo "🚀 Building frontend..."
cd frontend && npm run build && cd ..

echo "📦 Creating archives..."
tar -czf backend.tar.gz backend/src backend/package.json
backend/.env
tar -czf frontend-dist.tar.gz frontend/dist

echo "📁 Uploading to server..."
scp backend.tar.gz root@$1:/var/www/
scp frontend-dist.tar.gz root@$1:/var/www/

echo "✅ Deployment ready!"
echo "Next: SSH to server and extract files"
```

:Usage

```
chmod +x deploy-cloud.sh
./deploy-cloud.sh your-server-ip
```


Local Network Quick Deploy (Raspberry Pi)

save as: [deploy-local.sh](#)

```
#!/bin/bash

echo "🚀 Building..."
cd frontend && npm install && npm run build && cd ..
cd backend && npm install && cd ..

echo "📁 Copying to server..."
ssh pi@192.168.1.100 "rm -rf /opt/rf-learning"
scp -r . pi@192.168.1.100:/opt/rf-learning

echo "🔄 Restarting services..."
ssh pi@192.168.1.100 "pm2 restart rf-learning-backend"

echo "✅ Done! Access at http://192.168.1.100"
```

:Usage

```
chmod +x deploy-local.sh
./deploy-local.sh
```

סיכום

בחר את האפשרות שלך:

Cloud (עם אינטרנט): 🌐

1. DigitalOcean-דלוג ל (\$5/month)
2. above בצע את שלבים 1-7
3. אתה חי ב-1 שעה

Local Network (ללא אינטרנט): 🏠

1. Raspberry Pi קנה (\$55)
2. above בצע את שלבים 1-8
3. גישה ח-כל מכשיר ברשת
4. ללא עלויות חודשיות

Hybrid: 🎯

Cloud + Local Network
Best of both worlds!

עדכון: 2026-06-28

גרסה: 1.0

סטטוס: Ready for Production

ספר המערכת - לומדת ענף 71

עדכון אחרון: 2026-06-28

גרסה: Extended 1.0